Phase 6 – Explainable AI (SHAP & LIME)

Objective

Apply SHAP and LIME to interpret CNN intrusion detection predictions. Measure explanation time and compare interpretability trade-offs.

```
# ============================================================
# Install & Imports
# ============================================================

!pip install shap lime --quiet

import numpy as np
import torch
import torch.nn as nn
import shap
import time
import pandas as pd
import random
from lime import lime_tabular
from scipy.stats import spearmanr
import warnings
warnings.filterwarnings("ignore")

# ============================================================
# Random Seed
# ============================================================

seed = 42
torch.manual_seed(seed)
np.random.seed(seed)
random.seed(seed)

# ============================================================
# Model Definition
# ============================================================

class CNN1D(nn.Module):
    def __init__(self, num_classes=4):
        super(CNN1D, self).__init__()

        self.conv1 = nn.Conv1d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm1d(32)

        self.conv2 = nn.Conv1d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm1d(64)
```

```
        self.conv3 = nn.Conv1d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm1d(128)

        self.pool = nn.MaxPool1d(2)
        self.dropout = nn.Dropout(0.5)
        self.global_pool = nn.AdaptiveAvgPool1d(1)

        self.fc1 = nn.Linear(128, 64)
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x):
        x = self.pool(torch.relu(self.bn1(self.conv1(x))))
        x = self.pool(torch.relu(self.bn2(self.conv2(x))))
        x = self.pool(torch.relu(self.bn3(self.conv3(x))))

        x = self.global_pool(x)
        x = x.view(x.size(0), -1)

        x = self.dropout(torch.relu(self.fc1(x)))
        x = self.fc2(x)
        return x


# ============================================================
# Load Model
# ============================================================

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = CNN1D(num_classes=4).to(device)
model.load_state_dict(
    torch.load("/content/drive/MyDrive/nsl_kdd/cnn_model_final.pth",
               map_location=device)
)

model.eval()
print("Model loaded successfully.")

# ============================================================
# Load Data
# ============================================================

base_path = "/content/drive/MyDrive/nsl_kdd/"

X_test = np.load(base_path + "X_test.npy")
y_test = np.load(base_path + "y_test.npy")

X_test_tensor = torch.tensor(X_test,
dtype=torch.float32).unsqueeze(1).to(device)
```

```python
print("Test data shape:", X_test.shape)

# ============================================================
# SHAP ANALYSIS
# ============================================================

print("\n===== SHAP ANALYSIS =====")

background = X_test_tensor[:50]
samples = X_test_tensor[:100]

explainer = shap.GradientExplainer(model, background)

start_time = time.time()
shap_values = explainer.shap_values(samples)
shap_time = time.time() - start_time

print("Total SHAP computation time:", shap_time)

# ============================================================
# CORRECT SHAP SHAPE HANDLING FOR YOUR OUTPUT
# ============================================================

shap_values = np.array(shap_values)

print("Raw SHAP shape:", shap_values.shape)

# Expected shape from your run:
# (samples, 1, features, classes)

if shap_values.shape[1] == 1:
    shap_values = shap_values.squeeze(1)  # remove channel dimension

# Now shape should be:
# (samples, features, classes)

print("Processed SHAP shape:", shap_values.shape)

# Aggregate across classes for global explanation
shap_global = np.mean(np.abs(shap_values), axis=2)

# Aggregate across classes
shap_global = np.mean(np.abs(shap_values), axis=2)

features = samples.cpu().numpy().squeeze(1)

# Global summary plot
shap.summary_plot(shap_global, features, show=True)

# ============================================================
# Quantitative Global Importance
```

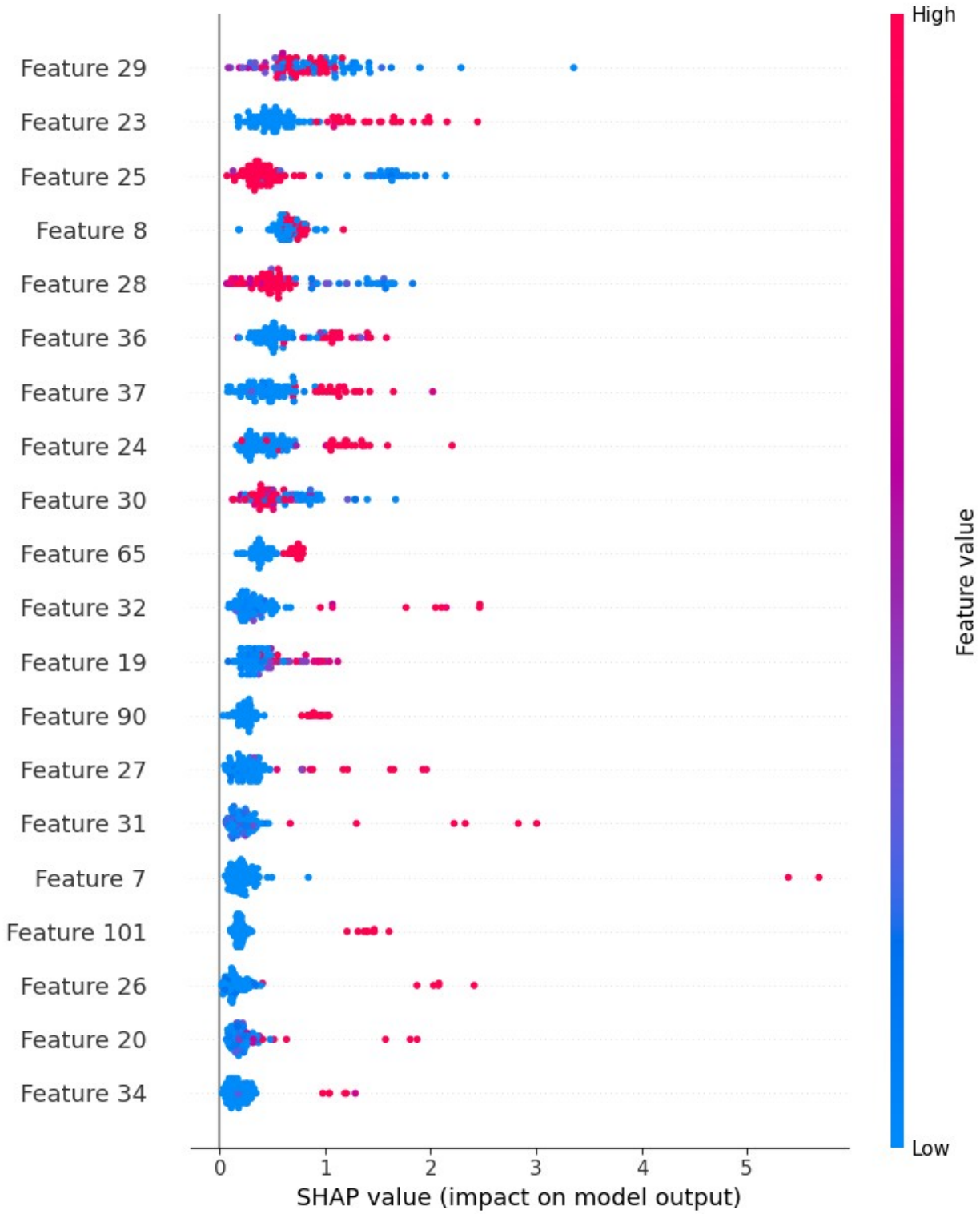
```python
# ============================================================

mean_abs_shap = np.mean(shap_global, axis=0)

top_indices = np.argsort(mean_abs_shap)[::-1][:10]

print("\nTop 10 Most Important Features:")
for rank, idx in enumerate(top_indices, 1):
    print(f"{rank}. Feature {idx} → {mean_abs_shap[idx]:.6f}")

global_importance_df = pd.DataFrame({
    "Feature_Index": np.arange(len(mean_abs_shap)),
    "Mean_Absolute_SHAP": mean_abs_shap
})

global_importance_df.to_csv(
    base_path + "shap_global_importance_phase6.csv",
    index=False
)

# ============================================================
# SHAP Stability Analysis
# ============================================================

print("\n===== SHAP STABILITY ANALYSIS =====")

num_runs = 5
top_k = 20

rankings = []
importance_runs = []

for run in range(num_runs):

    indices = np.random.choice(len(X_test_tensor), 100, replace=False)
    batch = X_test_tensor[indices]

    shap_vals = explainer.shap_values(batch)
    shap_vals = np.array(shap_vals)

    # Your SHAP format: (samples, 1, features, classes)
    if shap_vals.shape[1] == 1:
        shap_vals = shap_vals.squeeze(1)

    # Now shape: (samples, features, classes)

    # Aggregate across classes
    shap_global_run = np.mean(np.abs(shap_vals), axis=2)

    # Mean importance across samples
    mean_importance_run = np.mean(shap_global_run, axis=0)
```

```
    importance_runs.append(mean_importance_run)
    rankings.append(np.argsort(-mean_importance_run))



# ---- Spearman Correlation ----
spearman_scores = []
for i in range(num_runs):
    for j in range(i+1, num_runs):
        corr, _ = spearmanr(rankings[i], rankings[j])
        spearman_scores.append(corr)

mean_spearman = np.mean(spearman_scores)

# ---- Top-K Overlap ----
overlaps = []
for i in range(num_runs):
    for j in range(i+1, num_runs):
        top_i = set(rankings[i][:top_k])
        top_j = set(rankings[j][:top_k])
        overlap = len(top_i & top_j) / top_k
        overlaps.append(overlap)

mean_overlap = np.mean(overlaps)

# ---- Variance ----
importance_array = np.array(importance_runs)
mean_variance = np.mean(np.var(importance_array, axis=0))

print("Mean Spearman Rank Correlation:", mean_spearman)
print("Mean Top-K Feature Overlap:", mean_overlap)
print("Mean SHAP Importance Variance:", mean_variance)

stability_df = pd.DataFrame({
    "Metric": [
        "Mean Spearman Rank Correlation",
        "Mean Top-K Feature Overlap",
        "Mean SHAP Importance Variance"
    ],
    "Value": [
        mean_spearman,
        mean_overlap,
        mean_variance
    ]
})

stability_df.to_csv(
    base_path + "shap_stability_metrics_phase6.csv",
    index=False
```

```
)

# ============================================================
# LIME ANALYSIS
# ============================================================

print("\n===== LIME ANALYSIS =====")

explainer_lime = lime_tabular.LimeTabularExplainer(
    training_data=X_test,
    mode='classification'
)

def predict_fn(x):
    x_tensor = torch.tensor(x,
dtype=torch.float32).unsqueeze(1).to(device)
    with torch.no_grad():
        outputs = model(x_tensor)
        return torch.softmax(outputs, dim=1).cpu().numpy()

with torch.no_grad():
    preds = model(X_test_tensor)
    pred_labels = torch.argmax(preds, dim=1).cpu().numpy()

correct_idx = np.where(pred_labels == y_test)[0][0]
mis_idx = np.where(pred_labels != y_test)[0][0]

print("Correct sample index:", correct_idx)
print("Misclassified sample index:", mis_idx)

exp_correct = explainer_lime.explain_instance(
    X_test[correct_idx],
    predict_fn,
    num_features=10
)

exp_mis = explainer_lime.explain_instance(
    X_test[mis_idx],
    predict_fn,
    num_features=10
)

exp_correct.show_in_notebook()
exp_mis.show_in_notebook()

# ============================================================
# LATENCY ANALYSIS
# ============================================================

print("\n===== LATENCY ANALYSIS =====")
```

```python
# SHAP per-sample
shap_time_per_sample = shap_time / samples.shape[0]

# LIME per-sample (average over 5 runs)
lime_times = []
for i in range(5):
    start = time.time()
    explainer_lime.explain_instance(
        X_test[i],
        predict_fn,
        num_features=10
    )
    lime_times.append(time.time() - start)

lime_time_per_sample = np.mean(lime_times)

print("SHAP time per sample:", shap_time_per_sample)
print("LIME time per sample:", lime_time_per_sample)

latency_df = pd.DataFrame({
    "Method": ["SHAP", "LIME"],
    "Time_per_Sample_sec": [
        shap_time_per_sample,
        lime_time_per_sample
    ]
})

latency_df.to_csv(
    base_path + "explainability_latency_phase6.csv",
    index=False
)

latency_df

print("\n Phase 6 Complete Successfully.")
```

```
Model loaded successfully.
Test data shape: (22544, 122)

===== SHAP ANALYSIS =====
Total SHAP computation time: 44.772971868515015
Raw SHAP shape: (100, 1, 122, 4)
Processed SHAP shape: (100, 122, 4)
```




```
Top 10 Most Important Features:
1. Feature 29 → 0.858873
2. Feature 23 → 0.756565
```

```
3. Feature 25 → 0.700991
4. Feature 8 → 0.674481
5. Feature 28 → 0.671332
6. Feature 36 → 0.670352
7. Feature 37 → 0.616721
8. Feature 24 → 0.602104
9. Feature 30 → 0.572756
10. Feature 65 → 0.511060

===== SHAP STABILITY ANALYSIS =====
Mean Spearman Rank Correlation: 0.5324336381150019
Mean Top-K Feature Overlap: 0.945
Mean SHAP Importance Variance: 0.00033361248815575237

===== LIME ANALYSIS =====
Correct sample index: 0
Misclassified sample index: 3

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>


===== LATENCY ANALYSIS =====
SHAP time per sample: 0.44772971868515016
LIME time per sample: 1.3096715450286864

 Phase 6 Complete Successfully.
```

```python
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```